

GIẢI PHÁP THUẬT TOÁN: AURA TRACKER

Bài toán: Hệ thống giám sát Aura

Ngày 19 tháng 11 năm 2025

1 TỔNG QUAN BÀI TOÁN

Bài toán yêu cầu tính tổng lượng Aura của các đối tượng (Hunter) được phân bố trên n vùng theo dõi hình chữ nhật. Ràng buộc quan trọng nhất của bài toán là: mỗi Hunter chỉ đóng góp giá trị Aura của mình cho **vùng theo dõi có diện tích nhỏ nhất** chứa nó.

Các mức độ dữ liệu (Subtasks):

- **Subtask 1:** $N, M \leq 10^3$ (Dữ liệu nhỏ).
- **Subtask 2:** $W, H \leq 10^3$ (Kích thước bản đồ nhỏ, số lượng vùng và Hunter lớn).
- **Subtask 3:** $W, H \leq 10^9$ (Kích thước bản đồ rất lớn, trường hợp tổng quát).

2 PHÂN TÍCH CHI TIẾT GIẢI PHÁP

2.1 Subtask 1: $N, M \leq 10^3$

1. Phương pháp thiết kế thuật toán

Sử dụng phương pháp **Kiểm tra toàn diện (Brute Force)**.

a. Ý tưởng chủ đạo:

Với kích thước dữ liệu đầu vào nhỏ, ta có thể áp dụng chiến lược kiểm tra trực tiếp mọi khả năng. Cụ thể, mỗi Hunter sẽ được đối chiếu vị trí với toàn bộ danh sách các vùng theo dõi để xác định vùng chứa tối ưu nhất.

b. Quy trình thực hiện:

1. Khởi tạo mảng kết quả `Res` và biến `Out_of_zone` (tổng Aura không thuộc vùng nào).
2. Duyệt qua danh sách Hunter, với mỗi Hunter j :
 - Khởi tạo trạng thái tìm kiếm: $best_id = -1$, $min_area = +\infty$.
 - Duyệt qua toàn bộ N vùng theo dõi.
 - Kiểm tra điều kiện hình học: Nếu Hunter j nằm trong vùng i , thực hiện so sánh diện tích S_i với min_area . Nếu $S_i < min_area$, cập nhật $best_id = i$ và $min_area = S_i$.
 - Kết thúc vòng lặp vùng: Cộng giá trị Aura của Hunter vào vùng $best_id$. Nếu không tìm thấy vùng nào, cộng vào `Out_of_zone`.

c. Chứng minh tính đúng đắn: Thuật toán thực hiện vét cạn không gian mẫu (xét tất cả N vùng cho mỗi Hunter). Theo nguyên lý cực trị, giá trị cuối cùng của biến so sánh chắc chắn là giá trị nhỏ nhất toàn cục trong tập hợp các vùng chứa Hunter. Do đó, thuật toán đảm bảo độ chính xác tuyệt đối.

2. Lý do lựa chọn phương pháp

Với ràng buộc $N, M \leq 1000$, tổng số phép toán cơ bản là $N \times M = 10^6$. Các hệ thống chấm bài hiện đại có khả năng xử lý khoảng 10^8 phép tính/giây. Do đó, giải thuật này có thời gian thực thi khoảng $0.01s$, nằm trong giới hạn an toàn, đồng thời dễ dàng cài đặt để kiểm chứng kết quả.

3. Đánh giá độ phức tạp

- **Độ phức tạp Thời gian:** $O(N \cdot M)$ (Do sử dụng hai vòng lặp lồng nhau).
- **Độ phức tạp Không gian:** $O(N + M)$ (Lưu trữ dữ liệu đầu vào).

2.2 Subtask 2: $W, H \leq 10^3$

1. Phương pháp thiết kế thuật toán

Sử dụng chiến lược **Tham lam (Greedy)** kết hợp **Cấu trúc dữ liệu Cây Fenwick 2D**.

a. Ý tưởng chủ đạo:

Thay vì sử dụng các thuật toán hình học phức tạp để xác định vùng bao nhau, ta áp dụng chiến lược tham lam dựa trên diện tích: Xử lý các vùng theo thứ tự từ nhỏ đến lớn. Để hỗ trợ việc tính toán và cập nhật trạng thái bản đồ nhanh chóng, ta sử dụng **Fenwick Tree 2D (Binary Indexed Tree)**. Cấu trúc này cho phép thực hiện hai thao tác: Truy vấn tổng Aura trong một hình chữ nhật và Cập nhật giá trị tại một điểm với độ phức tạp Logarithmic.

b. Quy trình thực hiện:

1. **Khởi tạo:** Xây dựng Fenwick Tree 2D kích thước $W \times H$. Cập nhật toàn bộ vị trí và giá trị Aura của các Hunter vào cây.
2. **Sắp xếp:** Sắp xếp danh sách các vùng theo dõi theo thứ tự **Diện tích tăng dần**. Điều này đảm bảo các vùng nhỏ (vùng con) luôn được xử lý trước các vùng lớn (vùng cha).
3. **Xử lý tuần tự:** Duyệt qua danh sách vùng đã sắp xếp:
 - **Truy vấn (Query):** Tính tổng lượng Aura hiện có trong phạm vi hình chữ nhật của vùng i thông qua Fenwick Tree. Giá trị này chính là kết quả của vùng i .
 - **Cập nhật giảm trừ (Update):** Sau khi ghi nhận kết quả, thực hiện loại bỏ giá trị Aura của các Hunter thuộc vùng này khỏi Fenwick Tree (cộng giá trị âm). Thao tác này đảm bảo các vùng lớn hơn bao bên ngoài sẽ không tính lặp lại lượng Aura của các vùng con đã được xử lý.

c. Chứng minh tính đúng đắn: Do tính chất sắp xếp tăng dần về diện tích, tại thời điểm xét một vùng R , tất cả các vùng con nằm trong R (nếu có) đều đã được xử lý và lượng Aura của chúng đã bị loại bỏ khỏi cấu trúc dữ liệu. Vì vậy, kết quả truy vấn tại bước này chính xác là tổng Aura của những Hunter thuộc R nhưng không thuộc bất kỳ vùng con nào của nó.

2. Lý do lựa chọn phương pháp

Ràng buộc $W, H \leq 1000$ cho phép khởi tạo cấu trúc dữ liệu bảng 2D. Fenwick Tree 2D hỗ trợ các thao tác cập nhật và truy vấn trong thời gian $O(\log W \cdot \log H)$, hiệu quả hơn nhiều so với việc tái xây dựng bằng Prefix Sum ($O(W \cdot H)$) sau mỗi lần cập nhật. Chiến lược tham lam giúp đơn giản hóa logic xử lý bao hàm.

3. Đánh giá độ phức tạp

- **Độ phức tạp Thời gian:** $O(K \cdot \log W \cdot \log H + N \log N)$.
 - $N \log N$: Chi phí sắp xếp các vùng.
 - $K \cdot \log W \cdot \log H$: Chi phí cho K lần thao tác trên cây (với K tỉ lệ thuận với số lượng Hunter và Vùng).
- **Độ phức tạp Không gian:** $O(W \cdot H)$ (Bộ nhớ cho cây Fenwick 2D).

2.3 Subtask 3: $W, H \leq 10^9$ (Không giới hạn)

1. Phương pháp thiết kế thuật toán

Sử dụng phương pháp **Hình học tính toán - Kỹ thuật Sự kiện quét (Sweep Line)**.

a. Ý tưởng chủ đạo:

Do tọa độ không gian rất lớn (10^9), việc biểu diễn bản đồ dưới dạng lưới là bất khả thi. Ta áp dụng kỹ thuật Sweep Line để "rời rạc hóa" không gian. Một đường thẳng ảo sẽ quét dọc theo trục hoành (X), biến bài toán 2D tĩnh thành bài toán quản lý các đoạn thẳng động trên trục tung (Y).

b. Quy trình thực hiện:

1. **Tạo sự kiện:** Chuyển đổi dữ liệu đầu vào thành 3 loại sự kiện tọa độ trên trục X:

- *Event_ADD*: Tại cạnh trái X_1 của vùng $\rightarrow$ Thêm đoạn $[Y_1, Y_2]$ vào cấu trúc dữ liệu.
- *Event_REMOVE*: Tại cạnh phải X_2 của vùng $\rightarrow$ Xóa đoạn $[Y_1, Y_2]$ khỏi cấu trúc dữ liệu.
- *Event_QUERY*: Tại vị trí X_h của Hunter $\rightarrow$ Truy vấn tại tọa độ Y_h .

2. **Sắp xếp:** Sắp xếp toàn bộ sự kiện theo tọa độ X tăng dần.

3. **Quét và Xử lý:** Duyệt tuần tự các sự kiện. Sử dụng cấu trúc dữ liệu cây (Segment Tree hoặc Ordered Set) để quản lý tập hợp các đoạn Y đang hoạt động. Khi gặp sự kiện truy vấn, cấu trúc dữ liệu sẽ trả về định danh của vùng có diện tích nhỏ nhất đang phủ lên tọa độ Y_h .

c. **Chứng minh tính đúng đắn:** Tại thời điểm xử lý sự kiện Hunter, cấu trúc dữ liệu đảm bảo chứa thông tin chính xác của tất cả các lát cắt vùng đi qua hoành độ X hiện tại. Việc truy vấn trên cấu trúc dữ liệu đảm bảo tìm ra vùng bao phủ thỏa mãn tiêu chí diện tích nhỏ nhất nhờ vào tính chất thứ tự và phân cấp của cây.

2. Lý do lựa chọn phương pháp

Phương pháp này giải quyết được bài toán với tọa độ lớn bằng cách đưa độ phức tạp thuật toán phụ thuộc vào số lượng đối tượng (N, M) thay vì kích thước bản đồ (W, H). Đây là giải pháp chuẩn mực và tối ưu nhất cho lớp bài toán hình học này.

3. Đánh giá độ phức tạp

- **Độ phức tạp Thời gian:** $O((N + M) \log(N + M))$.
 - Chi phí chủ yếu đến từ việc sắp xếp sự kiện và các thao tác $O(\log N)$ trên cấu trúc dữ liệu cây.
- **Độ phức tạp Không gian:** $O(N + M)$ (Chỉ tốn bộ nhớ lưu trữ danh sách sự kiện, độc lập với tọa độ).